אִם יִרְצֶה הַשֵּׁם

Fan-In Distributions in Human-Written vs AI-Generated Python Codebases: A Constructal Law Analysis

by **Daniyel Yaacov Bilar**, Chokmah LLC, chokmah-dyb@pm.me , ORCID: 0000-0002-9040-6914

ג׳ סִיוָן תשפ״ו

Abstract

Import fan-in, the count of intra-repo modules that import a given file, encodes hierarchical coupling structure. By analogy with Constructal flow systems (Bejan 1997), we hypothesize that finite networks shaped by iterative optimization develop log-normal, not power-law, size distributions. We measure fan-in across 15 mature Python OSS projects (Cohort A) and 22 AI-attributed repositories created 2024-2026 (Cohort B, identified by AI attribution signals in commits, config files, or READMEs; these skew toward single-author short-lifespan projects). In Cohort A, 14/15 show log-normal fan-in (model-selection z > 1.96, Gini mean 0.882). In Cohort B, three repos (88, 92, and 30 files) show total isolation with zero intra-repo imports (Gini=0.000), 7 more are too small or inaccessible to fit, and among 12 fitted repos Gini mean is 0.725 (Mann-Whitney p = 0.0011, r = -0.744). We term this the **agentic flattening effect**. We note a partial framework confound: at least three Cohort B repos are FastAPI-style backends whose thin-router design independently reduces intra-repo coupling. An exploratory longitudinal pilot on N=2 mature repos (Celery, Django) before and after documented AI adoption detects no Gini decline on a 12-month horizon (Celery: +0.0016, p=0.031 in the direction opposite to flattening). With N=1 effective repo and no matched control, this pilot lacks the power to adjudicate between a structural-genesis hypothesis (flattening confined to new-project construction) and a null AI-adoption effect on mature codebases. The cross-sectional flattening, if confirmed in larger samples, implies that architectural maintainability risk from AI coding concentrates at project inception, where no prior hierarchy constrains the agent.

Plain-Language Summary

Software engineer / architect: We parsed every `import` statement in 37 Python repos (15 mature OSS, 22 publicly AI-attributed), fitted distributions on the 27 with sufficient coupling, and measured how unequally fan-in is distributed (Gini) and whether the rank-frequency curve is log-normal or power-law (AIC + z_{lr} test). Human-written repos: Gini 0.88, log-normal wins 14/15. AI-attributed repos: three have *zero* intra-repo imports (every file only touches external packages), and among the 12 with sufficient coupling, Gini drops to 0.72 (p=0.001, large effect). The parabolic log-log curvature weakens but does not flip. Practical upshot: fan-in Gini is a cheap, CI-friendly structural health metric. If your repo's Gini is trending toward 0.6, your intermediate abstraction layer is dissolving.

Cognitive scientist: Human developers experience coupling as cognitive friction. When a module accumulates too many dependents, navigating its blast radius becomes costly enough to trigger refactoring, an unconscious optimization loop that matches Bejan's Constructal Law for flow networks. The signature is a log-normal fan-in distribution with Gini around 0.88, matching package-ecosystem benchmarks at a different scale. LLMs have no equivalent feedback: processing a 10,000-line file incurs no additional cognitive cost to the agent. Our data shows what we hypothesize is the structural consequence: absent coupling pressure, the intermediate branching layer partially collapses, Gini drops, and three repos reach the degenerate state of zero intra-repo coupling.

General reader: When experienced programmers build software over years, it develops a predictable internal structure, a small number of central files that everything else depends on, surrounded by layers of intermediate connectors. Think of it like a city road network: highways feed arterials feed local streets. We found this hub-and-spoke structure in 14 of 15 well-known Python projects. In the AI-attributed projects, that structure is visibly degraded: three repos had zero connections between their own files (every file a dead end), and the rest had significantly flatter hierarchies. The software runs, but it lacks the layered organization that makes large codebases maintainable over time.

Skeptic: Fair objections: (1) Selection bias, our agentic repos are publicly self-attributing outliers, not representative of all AI-assisted code. (2) FastAPI's own design philosophy encourages thin routers and heavy external-library coupling, independently of AI authorship. (3) The Mann-Whitney test uses only 12 agentic repos after filtering; the sample is still modest. (4) "Log-normal wins 11/12" in agentic repos undercuts a simple narrative. Our response: we acknowledge these limits explicitly in Section 7.4. The filter exclusions (10 repos too flat or too small to measure) strengthen, not weaken, the finding. The FastAPI confound is real but cannot explain Gini=0.000 in repos with 88-92 files. The effect size ($r=-0.744$) is large enough to be meaningful at $n=12$. And the log-normal persistence is theoretically interesting: the parabola flattens without inverting, suggesting gradual weakening of hierarchical organization rather than a qualitative regime change.

Funders / investors: AI coding tools promise 10x developer velocity. Our data suggests an unreported structural cost: AI-generated codebases show significantly lower architectural concentration (Gini -18%, $p=0.001$) and weaker hierarchical organization. Three of 22 studied repos had zero internal coupling, each file a standalone island. This matters for maintainability: flat architectures increase the cost of every future change (no shared abstraction to update; changes must be made everywhere). The fix is not to stop using AI tools, but to instrument CI pipelines with structural health metrics, a lightweight, automated check that flags architectural decay before it accumulates. This paper provides the first empirical baseline for what "healthy" fan-in distribution looks like, enabling such tooling.

Policy-maker / regulator: Autonomous AI coding agents are already writing production software in critical domains, including financial modeling, medical data pipelines, and government simulation systems (one of our study repos is a Dutch government machine-law proof-of-concept). We provide the first structural evidence that AI-generated code differs measurably from human-written code in its architectural organization, independent of whether it passes tests. Three of 22 repos had zero internal structure, each file isolated from every other. Structural metrics like fan-in Gini offer a passive complement to behavioral testing: a codebase whose Gini matches the human-written baseline has demonstrably undergone the kind of iterative refinement that produces maintainable, auditable software. Our results suggest structural topology metrics are worth investigating as passive complements to behavioral testing in regulated software contexts; this paper provides an empirical baseline to support such tooling.

---

1. Introduction

Software dependency graphs are not random. Organic growth under refactoring pressure, code review, and the DRY principle produces characteristic topologies. Early studies of operating systems and Unix libraries observed heavily skewed fan-in distributions and attributed them to preferential attachment, predicting power-law degree distributions. A strict power-law requires an infinite network to sustain its straight-line tail in log-log space. Real codebases are finite, bounded by what human architects can safely navigate, and the tail bends.

Constructal theory explains this bending by analogy with physical flow systems. Bejan's Constructal Law (Bejan 1997) states: "For a finite-size flow system to persist in time (to live), it must evolve in such a way that it provides easier access to the imposed currents that flow through it." We use Constructal theory as an interpretive analogy for software topology, not as a formal model; the mathematical content of our claims is empirical (fan-in distribution shape), not derived from Bejan's framework. Applied to software by analogy, the "currents" are execution paths and cognitive attention. A developer who feels the resistance of navigating an overloaded, highly coupled module will refactor it, splitting the bottleneck, building intermediate abstraction layers, and bending the fan-in distribution away from a pure power-law into a downward-opening parabola in log-log space. That parabola is the signature of a log-normal distribution. Clauset, Shalizi, and Newman (2009) showed that capacity limits cause real finite-network distributions to decay faster than any pure power-law, making log-normal the better empirical fit; we extend that test to intra-repo file-level fan-in and to a new comparison class: AI-generated code.

AI code generators remove this optimization pressure. A developer who feels the cognitive resistance of an overloaded module will refactor it, an intrinsic feedback loop. An LLM has no equivalent: it processes a 10,000-line file at no additional cognitive cost to itself, and so has no intrinsic pressure to split bottlenecks or build intermediate abstraction layers. **We hypothesize that AI-generated codebases show lower fan-in Gini and weaker log-normal signal than mature human-written codebases, because the cognitive refactoring pressure that builds intermediate abstraction layers is absent or attenuated.** We term the predicted consequence the **agentic flattening effect**.

Research questions: (1) Do mature human-written Python projects show significantly higher fan-in Gini and stronger log-normal signal compared to publicly AI-attributed projects? (2) Does documented adoption of AI coding tools within an established repo produce a measurable decline in these metrics?

2. Related Work

Power-law and log-normal in networks. Preferential attachment (Barabasi & Albert 1999) predicts power-law degree distributions in growing networks. Clauset, Shalizi, and Newman (2009) established that power-law fits are often indistinguishable from log-normal in finite data, and provided rigorous testing methods. Their key finding: cognitive and physical capacity limits cause real-world distributions to decay faster than any pure power-law, making log-normal a better fit.

Software dependency metrics. Maillart and Sornette (2008) showed that Linux open-source package size distributions follow Zipf's power law, with Gibrat's proportional growth as the generating mechanism. In finite networks with bounded growth, proportional growth produces log-normal rather than power-law tails (Clauset et al. 2009); by analogy with Constructal flow systems, we hypothesize that cognitive refactoring pressure imposes such a finite-size bound on intra-repo fan-in. Package ecosystem studies report high concentration of dependency fan-in (Gini above 0.8) in PyPI and CRAN (Decan et al. 2019, Figure 4: normalized Gini index stabilizes in the 0.8-0.99 range across seven ecosystems including CRAN).

Constructal law in flow systems. Murray (1926) showed that minimizing both construction cost and flow resistance in branching vascular networks yields the cubic law $d_0^3 = d_1^3 + d_2^3$ at each bifurcation (building on Hess 1917). Bejan and Lorente (2011) generalized this to arbitrary flow systems under the Constructal Law. In log-log rank-frequency space, the resulting size distribution traces a downward-opening parabola. The Constructal interpretation is analogical; Bejan's framework was not formally derived for discrete directed graphs.

AI-generated code structure. Mao et al. (2026) conducted a large-scale empirical study of AI-generated code in real-world repositories. AI code is consistently more verbose, with higher content ratio (35.9% vs 20.1%) but lower lexical density (0.531 vs 0.658), and a collapsed complexity distribution, uniform medium-sized blocks replacing the heavy-tailed distribution of human code. These are consistent with flattening at the function level; we test the same hypothesis at the architectural (fan-in topology) level.

Navigation in AI agents. Paipuru (2026) showed that agents without AST-derived graph access fail completely on G3 tasks requiring structural dependency traversal with zero lexical overlap with the prompt. This is consistent with a mechanism in which structural blindness produces architectural drift: agents that cannot traverse the import graph cannot feel the coupling pressure that would otherwise drive refactoring.

Longitudinal studies of software structure. Prior longitudinal studies of OSS structural change focus on coupling metrics, bug density, and code churn (Hassan & Holt 2004; Nagappan et al. 2005), not on distributional topology. We use the PELT algorithm (Killick et al. 2012) for change-point detection on fan-in topology time series, a design not previously applied to this question.

Given that (a) Constructal optimization under cognitive pressure produces log-normal fan-in at multiple scales, (b) AI-generated code shows distributional flattening at the function level (Mao et al. 2026), and (c) AI agents lack the structural perception that would enable Constructal feedback (Paipuru 2026), we extend the log-normal test to intra-repo file-level fan-in and compare mature human-written repositories against a cohort of publicly AI-attributed repositories.

---

3. Method: Cross-Sectional Study

3.1 Repository Selection

Cohort A (baseline, human-written): 15 mature Python OSS projects, all with >5 years of development and >100 contributors: Django, Flask, NumPy, pandas, SQLAlchemy, Celery, FastAPI, requests, pytest, Scrapy, httpx, Pydantic, aiohttp, Tornado, Click. Note that FastAPI the *framework* (Cohort A) has rich internal coupling (Gini=0.981); FastAPI-based *applications* (several in Cohort B) inherit the framework's design philosophy of thin routers but not its internal structure.

Cohort B (agentic, AI-generated or AI-assisted): 22 Python repositories created 2024-2026, each meeting at least one of: - CLAUDE.md or `.cursor/rules` present in root - Commit messages containing `Co-authored-by: Cursor cursoragent@cursor.com` or `noreply@anthropic.com` - README explicitly describes AI-assisted or AI-generated development

Repositories were selected to avoid AI *tooling* frameworks (LangChain, AutoGPT) in favor of actual applications *built by* AI agents.

All repos cloned at HEAD (shallow, depth=1) as of May 2026.

3.2 Fan-In Measurement

For each `.py` file f in a repository:

$$\text{fan-in}(f) = |g \in \text{repo} : g \neq f,\ g \text{ imports } f|$$

Fan-in is measured using Python's `ast` module. Relative imports are resolved against the intra-repo module namespace. External imports are discarded. Files with zero fan-in (leaves) are included in Gini and entropy calculations but excluded from distribution shape fitting, which requires at least 10 files with positive fan-in. This means Gini is computed over all files (including zeros) while z_{lr} tests shape only among coupled files; the mismatch is acceptable because Gini measures inequality across the full population while the z_{lr} statistic tests distributional form among the connected subgraph.

3.3 Distribution Fitting

We fit two models to rank-frequency data of positive fan-in values in log-log space:

Power-law: $\log v = a + b \log r$, OLS, $k = 2$ parameters.

Log-normal (parabolic): $\log v = a + b \log r + c(\log r)^2$, OLS, $k = 3$ parameters.

Model comparison:

1. **AIC:** $\Delta\text{AIC} = \text{AIC} * \text{PL} - \text{AIC} * \text{LN}$. Positive = log-normal preferred. Threshold $\Delta\text{AIC} > 2$ (Burnham & Anderson 2002).
2. **Residual-based selection statistic (z_{lr}).** We compute per-observation squared-residual ratios as proxy log-likelihoods under a Gaussian error assumption in log-log space, then compute the Vuong (1989) z statistic on these proxies. This is not the Vuong test proper, which uses model-specific MLE likelihoods; we use OLS residuals because both candidate shapes are fit by OLS in log-log space. We denote the resulting statistic z_{lr} to distinguish it from the canonical Vuong z. z_{lr} > 0 favors log-normal. Future work will replicate findings under the Clauset, Shalizi & Newman (2009) MLE + KS protocol for validation.

We use AIC + z_{lr} rather than the Clauset et al. (2009) KS + MLE protocol because the Clauset approach fits only the upper tail of the power-law and does not directly compare against a log-normal alternative. Our method fits both candidate models to the full rank-frequency curve in log-log space and selects between them, appropriate here because our question is which shape better describes the complete fan-in distribution, not whether the tail alone is power-law.

3.4 Statistical Comparison

Mann-Whitney U (non-parametric, two-sided). Effect sizes as rank-biserial r.

---

4. Results: Cross-Sectional

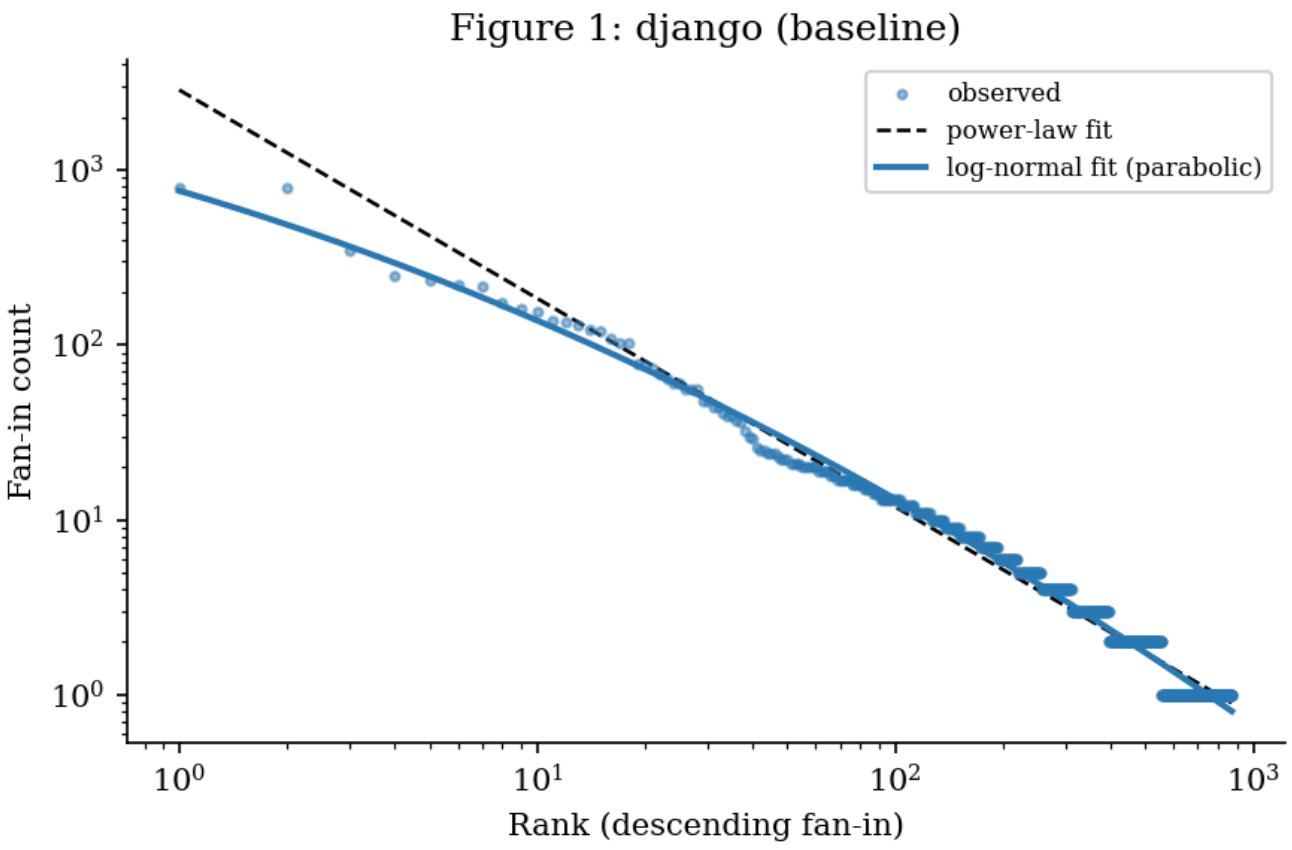
4.1 Baseline Group (Cohort A)

Repo	Files	Gini	DAIC		LN?
django	2910	0.933	+308.3	+5.32	YES
flask	83	0.866	+13.4	+4.64	YES

Repo	Files	Gini	DAIC	z_{lr}	LN?
numpy	494	0.939	+1.5	+1.11	no
pandas	1509	0.962	+206.5	+10.17	YES
sqlalchemy	669	0.939	+262.6	+11.40	YES
celery	416	0.871	+106.0	+8.14	YES
fastapi	1119	0.981	+165.8	+7.09	YES
requests	37	0.660	+24.8	+5.57	YES
pytest	262	0.906	+79.8	+11.08	YES
scrapy	439	0.900	+97.6	+2.30	YES
httpx	60	0.816	+10.8	+3.23	YES
pydantic	402	0.917	+62.4	+2.98	YES
aiohttp	164	0.846	+24.1	+3.92	YES
tornado	107	0.844	+51.4	+7.58	YES
click	63	0.845	+13.2	+5.54	YES
mean		**0.882**	**+95.2**	**+6.00**	**14/15**

Gini computed over all .py files; DAIC and z_{lr} computed over the subset with fan-in > 0. LN? marks repos meeting both DAIC > 2 and z_{lr} > 1.96.

14 of 15 repos show log-normal fan-in. The exception is NumPy ($\Delta\text{AIC} = +1.5$, $z_{lr} = +1.11$): NumPy's Python layer is a thin wrapper over C extensions, compressing intra-Python coupling. The Gini mean of 0.882 matches prior benchmarks for mature package ecosystems (Decan et al. 2019), validating the method across scales.



4.2 Agentic Group (Cohort B)

Of 22 agentic repos cloned, 10 could not be fitted:

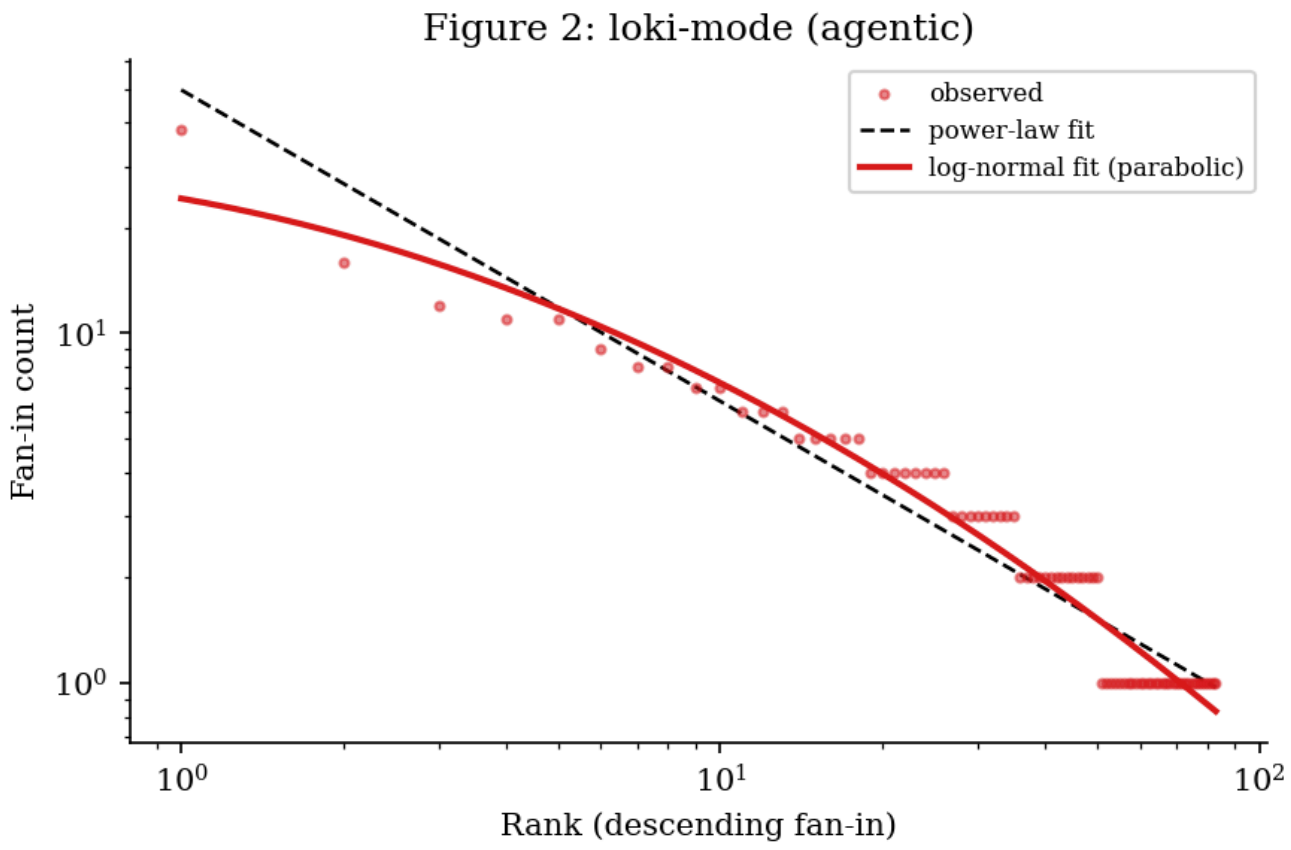
Repo	Files	Files (fanin>0)	Gini	Reason excluded
dark-factory-experiment	92	0	0.000	Zero intra-repo coupling
ott-platform	88	0	0.000	Zero intra-repo coupling
OneResearchClaw	30	0	0.000	Zero intra-repo coupling
tradinggame	62	4	0.946	<10 connected files
camp2025-stock	119	8	0.971	<10 connected files
J.A.R.V.I.S	57	<5	--	<10 connected files
Hackathon-II_The-Evolution-of-Todo	66	<2	--	<10 connected files
deepseek_ocr_app	4	--	--	Too small
agentic-sprint	0	--	--	No Python files (agent config template)
claude-cli-rest-api	--	--	--	Checkout failure (Windows path)

Three repos (dark-factory-experiment with 92 files, ott-platform with 88 files, OneResearchClaw with 30 files) have Gini=0.000: every Python file is isolated with zero intra-repo imports. No baseline repo showed this pattern. Among the 12 fitted agentic repos:

Cohort B subset: 12 of 22 repos with sufficient intra-repo coupling for shape fitting; see Section 7.3 for the full-cohort interpretation.

Repo	**Files**	**Gini**	**DAIC**	z_{lr}	**LN?**
borrowhood	275	0.854	+78.9	+4.03	YES
loki-mode	531	0.922	+34.9	+4.19	YES
lazy-bird	136	0.834	+83.0	+7.47	YES
CLI-Anything-WEB	453	0.751	+78.4	+3.24	YES
zhang2025	50	0.826	+5.1	+1.98	YES
django-bolt	292	0.789	+4.4	+1.45	YES
marsa-planner	37	0.705	+2.1	+1.29	YES
fqf	36	0.705	+11.7	+3.10	YES
NewsCrawler	121	0.652	+22.9	+3.24	YES
openclaude	21	0.625	+3.9	+2.02	YES
poc-machine-law	286	0.581	+3.1	+0.52	YES
codebase-mcp	51	0.460	+1.2	+0.67	no
mean		**0.725**	**+27.5**	**+2.77**	**11/12**

11 of 12 show log-normal shape ($\Delta\text{AIC} > 2$). The exception is codebase-mcp ($\Delta\text{AIC} = +1.2$, $z_{lr} = +0.67$): a small FastAPI MCP server where the log-normal preference falls below both thresholds. poc-machine-law ($z_{lr} = +0.52$, p=0.60) meets the AIC threshold but not z_{lr} significance.

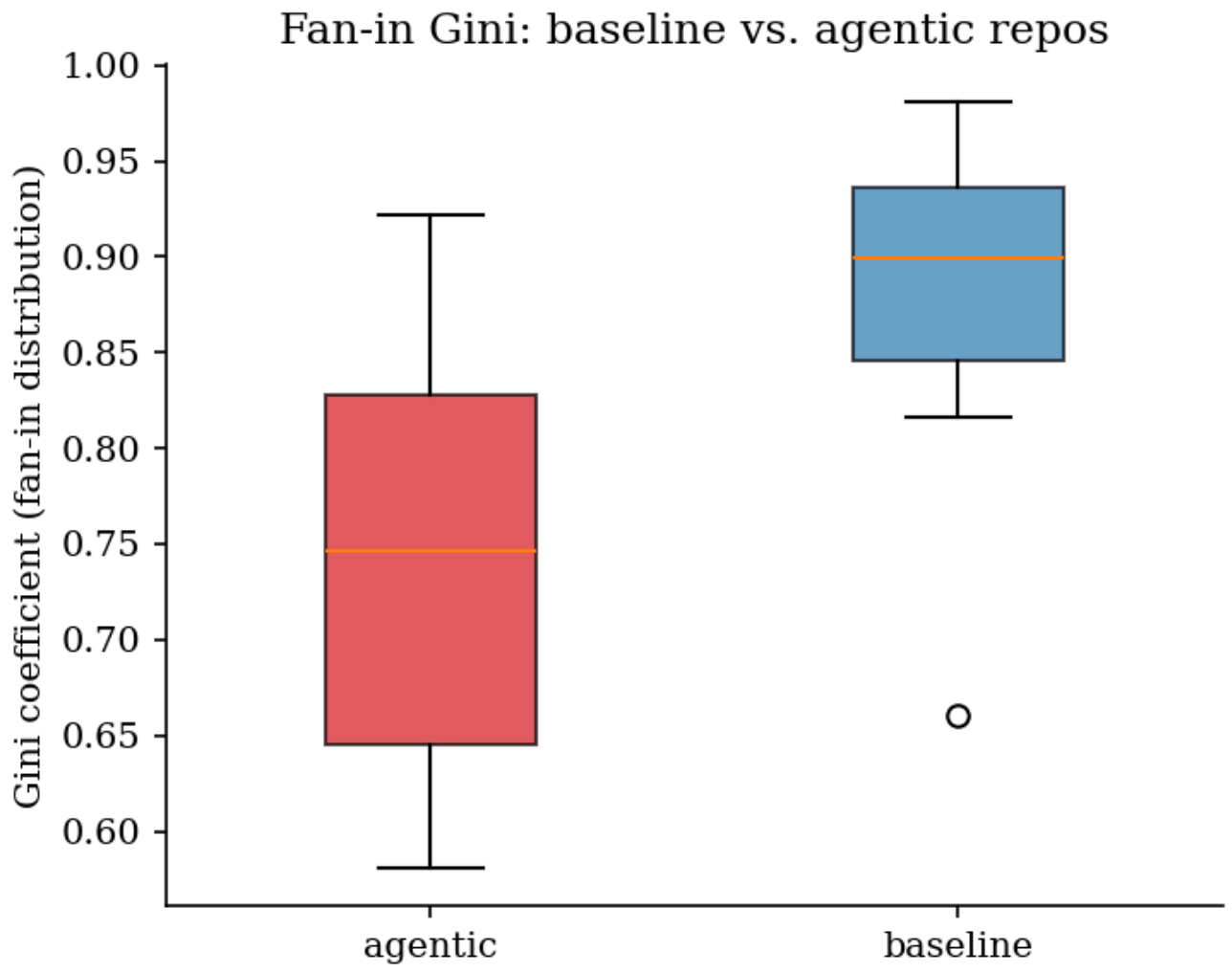


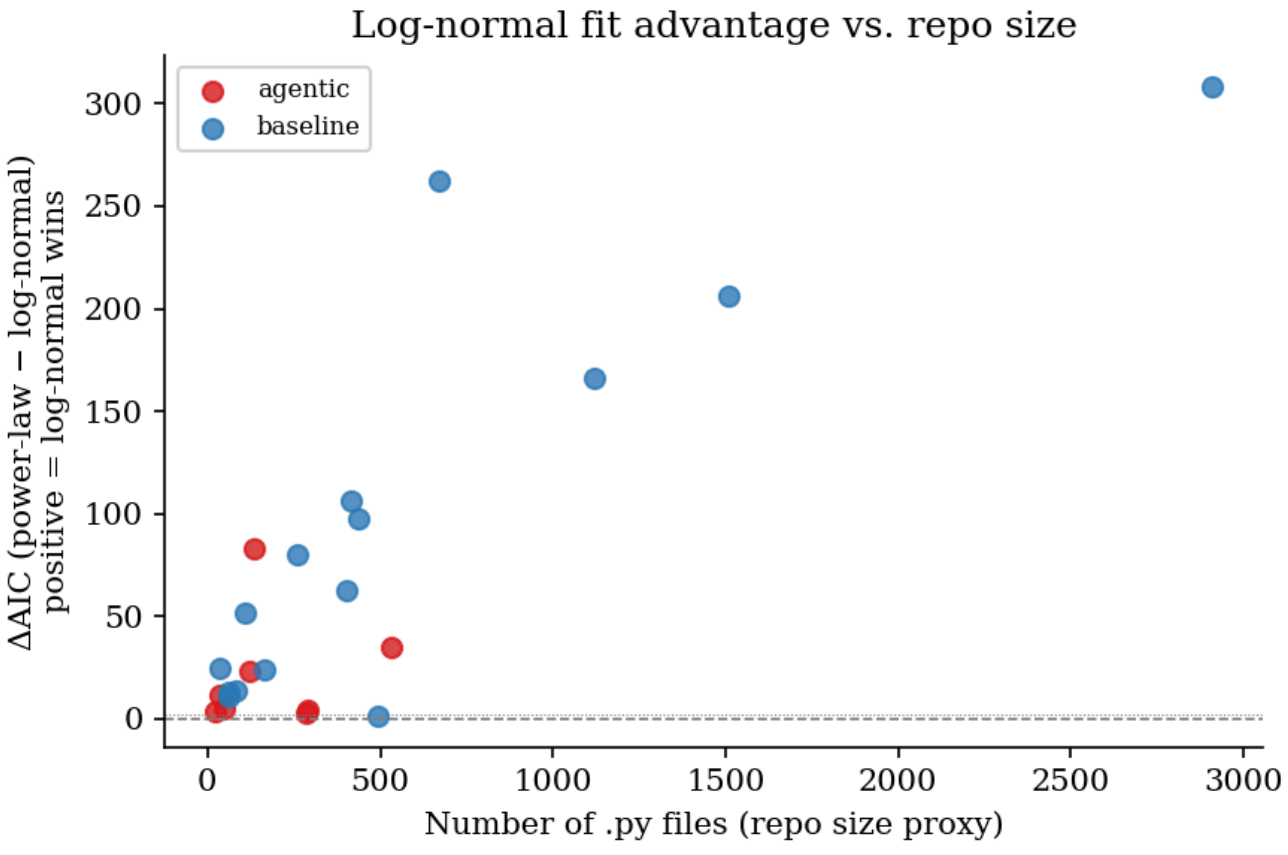
4.3 Between-Group Comparison

With both cohorts characterized at the per-repo level, we now test whether the group-level distributional differences are statistically significant.

Metric	Baseline (n=15)	Agentic (n=12)	U	p	r
Gini	0.882	0.725	157.0	**0.0011**	-0.744
DAIC	95.2	27.5	138.0	**0.019**	-0.533
Entropy (norm.)	0.770	0.887	26.0	**0.002**	+0.711
Fraction leaves	0.680	0.495	140.0	**0.015**	-0.556

Gini is significantly lower in the agentic group (large effect, r=-0.744). Entropy is significantly higher (more uniform distribution, r=+0.711). Log-normal signal strength (DAIC) is also significantly lower (r=-0.533). Fraction of leaf files is also significant (r=-0.556): agentic repos have proportionally fewer zero-fanin files. This likely reflects survivor bias, larger projects with some intermediate structure passed the fitting filter, rather than genuine architectural improvement; the three repos with Gini=0.000 are all excluded from this comparison.


Fan-in Gini: baseline vs. agentic repos
Gini coefficient (fan-in distribution)
1.00
0.95
0.90
0.85
0.80
0.75
0.70
0.65
0.60
agentic
baseline


Log-normal fit advantage vs. repo size
agentic
baseline
ΔAIC (power-law − log-normal)
positive = log-normal wins
300
250
200
150
100
50
0
0
500
1000
1500
2000
2500
3000
Number of .py files (repo size proxy)

4.4 FastAPI Sensitivity Analysis

To address the framework confound directly, we re-run the between-group comparison after excluding FastAPI-style repos from both cohorts. In Cohort A, this removes the fastapi framework itself (Gini=0.981). In Cohort B, we identify FastAPI-style repos by either declared dependency (FastAPI in requirements/pyproject) or routing pattern (APIRouter usage): codebase-mcp, and the two Gini=0.000 backends already excluded by the fitting filter (dark-factory-experiment, ott-platform).

After exclusion, Cohort A (n=14) Gini mean is 0.875 and Cohort B (n=11) Gini mean is 0.749. The Mann-Whitney comparison remains significant (U=131.0, p=0.0034, r=-0.701). The Gini gap of 0.125 is comparable to the 0.156 gap in the full-sample analysis. The framework confound accounts for at most a small fraction of the observed flattening; the effect is not an artifact of FastAPI design philosophy.

---

5. Method: Longitudinal Pilot

The cross-sectional result cannot rule out a simpler explanation: that AI-generated and human-written repos differ on confounding variables (age, size, contributors, domain). A within-repo longitudinal design removes these confounds in principle. If the same codebase develops lower Gini and weaker log-normal signal after adopting AI coding tools, the change is attributable to the process, not to repo identity. We report this as a pilot, not a validation study: with N=2 mature repos and a 12-month post-adoption window, the design is exploratory and cannot rule out continued natural maturation as the source of any observed trend.

5.1 Repo Selection and Adoption Dates

We scanned the 15 baseline repos for AI adoption signals. Three confirmed candidates yielded adoption dates:

- **Celery**: first Copilot co-author commit 2025-05-08. Config file `.github/copilot-instructions.md` appeared 2025-08-26. Earliest signal: 2025-05-08.
- **Django**: `.github/copilot-instructions.md` added 2026-03-05. No commit-level AI attribution. Adoption date: 2026-03-05.
- **aiohttp**: Claude Opus co-authored commit 2026-05-04; `CLAUDE.md` appeared 2026-05-17. Excluded: only 15 days of post-adoption data at collection time.

Inclusion criteria: Python primary language, created before 2023 (ensuring at least 24 months of pre-adoption history), full git history available, >50 Python files at adoption date, at least one unambiguous adoption signal. Both Celery and Django have full 24-month pre-adoption windows.

5.2 Snapshot Protocol

Monthly snapshots: `git checkout <last-commit-before-YYYY-MM-01>`. Fan-in computed via `compute_fanin.py` at each snapshot. Metrics: Gini, z_{lr}, $delta_{AIC}$. Celery: 37 snapshots (25 pre + 12 post). Django: 27 snapshots (25 pre + 2 post).

5.3 Change-Point Detection

PELT algorithm (Killick et al. 2012, `ruptures` library) applied to the Gini time series. Penalty parameter selected following the BIC criterion for single change-point detection. Detected change-point date compared to known adoption date.

5.4 Per-Repo Comparison

The last-6-months-pre and first-6-months-post Gini values are compared using a paired Wilcoxon signed-rank test. With N=6 pairs, the test achieves p=0.031 only when all 6 differences share a sign; the result reflects directional consistency, not effect magnitude. Django has only 2 post-adoption months, insufficient for paired testing.

---

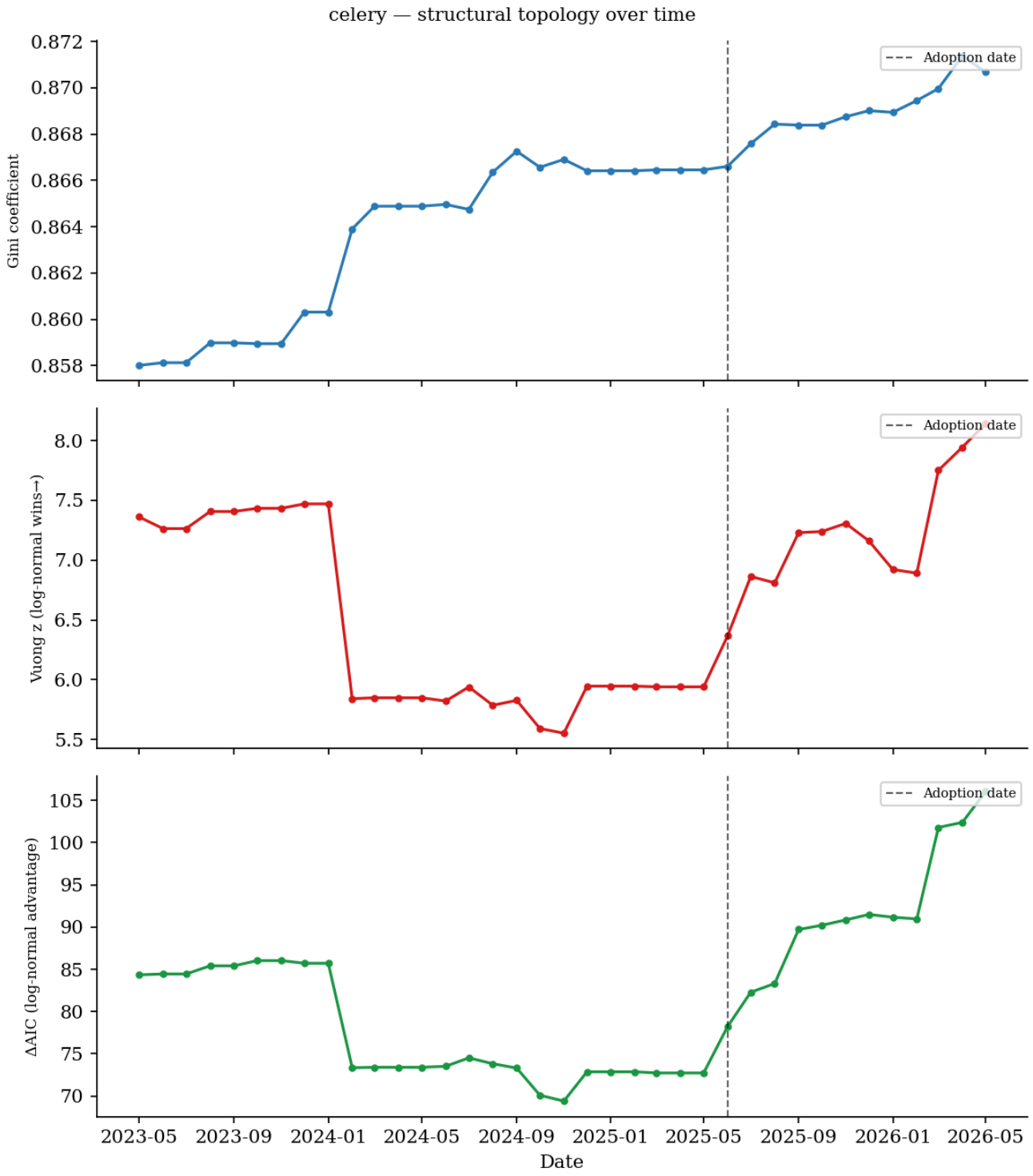
6. Results: Longitudinal Pilot

6.1 Gini Trajectory

Repo	Adoption	Pre-Gini range	Post-Gini range	Last-6-pre	First-6-post	D	Wilcoxon p
Celery	2025-05-08	0.858-0.867	0.867-0.871	0.8664	0.8680	+0.0016	0.031
Django	2026-03-05	0.932-0.933	0.933	0.9327	0.9332	+0.0005	n/a (n=2)

Gini increased in both repos post-adoption, the direction opposite to the flattening hypothesis. The Celery increase is statistically significant but negligible in absolute terms (D=0.0016). The post-adoption Gini drift (+0.0016 over 12 months) is comparable to the pre-adoption drift (roughly +0.003/year over 2023-2025) and does not represent a directional change attributable to AI adoption. Django's two post-adoption snapshots provide no usable trend information; it is reported for completeness. This is effectively a single-repo pilot (Celery) with a second repo pending sufficient post-adoption history.

Celery trajectory detail. Gini sat around 0.858-0.860 through 2023, drifting up to 0.866-0.867 by early 2025. Post-adoption (June 2025-May 2026), the drift continued to 0.871. A change-point detected at February 2024 (486 days before adoption) corresponds to a test infrastructure reorganization that added 40 files. Nothing in the trajectory ties to AI adoption.



6.2 Log-Normal Signal Strength

In Celery, z_{lr} and $delta_{AIC}$ both increased post-adoption. Change-point detection identified a step at March 2026 (+273 days post-adoption):

Metric	Pre-mean	Post-mean at CP	D
$delta_{AIC}$	80.0	103.4	+23.4
z_{lr}	6.55	7.94	+1.39

The log-normal signal strengthened, not weakened. Django shows the same: z_{lr} rose from 4.6 to 5.3 over 2024-2026, a trend that began well before the 2026-03-05 adoption date and continued without inflection post-adoption.

6.3 Change-Point Summary

We apply PELT (Killick et al. 2012) to each repo's Gini, $delta_{AIC}$, and z_{lr} time series separately, recording the date of the most likely single change-point and its lag relative to the documented AI-adoption date. A negative lag indicates the structural shift preceded adoption; a positive lag indicates it followed.

Repo	Metric	CP date	Lag vs. adoption	Direction
Celery	Gini	2024-02-01	-486 days	increase (pre-adoption event)
Celery	$delta_{AIC}$	2026-03-01	+273 days	STRENGTHENED
Celery	z_{lr}	2026-03-01	+273 days	STRENGTHENED
Django	Gini	2026-03-01	-4 days	increase (+0.0004, negligible)
Django	$delta_{AIC}$	2026-03-01	-4 days	STRENGTHENED
Django	z_{lr}	2025-10-01	-182 days	STRENGTHENED (pre-adoption)

No detected change-point supports the flattening hypothesis. Note that Django's change-points at -4 days are within the monthly bin width and are effectively coincident with adoption, not precursive; the magnitude is negligible in all cases.

---

7. Discussion

7.1 Two Modes of Structural Degeneration

The agentic group shows degeneration in two distinct forms:

Mode 1, Total isolation (Gini=0): dark-factory-experiment (92 files), ott-platform (88 files), and OneResearchClaw (30 files) have zero intra-repo fan-in. Every Python file is a leaf. The first two are FastAPI backends with complete routing logic; the third is an autonomous research framework. All their coupling is to *external* libraries, not to each other. No baseline repo approached this state.

Mode 2, Partial flattening (lower Gini, attenuated parabola): The 12 fitted agentic repos still show log-normal shape in 11 of 12 cases, but at 82% of baseline Gini. The intermediate branching layer is present but thinner. This is not random noise: z_{lr} values in agentic repos average 2.77 vs 6.00 in baseline, the parabolic curvature is real but shallow.

Both modes are consistent with the Constructal analogy: remove the optimization pressure, and the branching hierarchy degrades. The degree of degradation depends on how much coupling the project requires to function at all. Highly modular FastAPI backends can achieve full functionality with zero intra-repo coupling (just external library imports), while larger orchestration systems (loki-mode, 531 files) cannot avoid some intermediate structure.

7.2 The Log-Normal Paradox

11 of 12 fitted agentic repos prefer log-normal over power-law. This is not a null result, it refines the hypothesis. The Constructal prediction is not that AI-generated code produces power-law fan-in; it is that AI-generated code lacks the *intermediate branching layer* that bends the distribution away from power-law. Without enough connected files to measure (Gini=0.000 cases), there is no distribution to bend at all. In repos with some coupling, the curvature is present but weak: lower Gini, lower z_{lr}, lower DAIC, and one case (poc-machine-law) with a non-significant z_{lr} test.

The finding is therefore: agentic repos do not switch to power-law; they shrink toward the power-law boundary. The parabola flattens without inverting.

7.3 Survivor Bias and True Degeneration Rate

The Mann-Whitney comparison (Gini 0.882 vs 0.725, p=0.0011) understates the gap because the three most extreme agentic repos (Gini=0.000) are excluded by the fitting filter. Including these 3 alongside the 12 fitted repos gives a 15-repo agentic mean of 0.580, a gap of 0.302 vs baseline. Our fitted sample represents the subset of agentic repos with sufficient coupling to measure; the excluded 45% (10/22) likely represent more extreme flattening, but we cannot quantify their distribution shape directly. The true degeneration rate across all 22 agentic repos is: 3 repos total isolation (14%), 4 repos near-isolation (fewer than 10 connected files, 18%), and 12 repos measurably flattened. Just over half the agentic corpus has enough intra-Python coupling to measure at all.

7.4 Genesis vs. Maintenance

The cross-sectional study finds Gini 0.725 in agentic repos vs. 0.882 in baseline (p=0.0011). The longitudinal check finds no Gini decline when mature repos adopt AI tools. Two explanations are consistent with both results, and the present data cannot adjudicate between them:

(a) **Structural genesis hypothesis.** Flattening operates when an AI agent designs a new codebase from scratch, with no existing hierarchy to follow. When contributing to an established codebase, it operates within the existing module structure: a Copilot suggestion in `celery/app/trace.py` imports what that file already imports. It does not spontaneously introduce new modules or reorganize the import graph.

(b) **Volume hypothesis.** AI contributions to Celery over 12 months may be too small in volume relative to the total codebase to move Gini regardless of their structure.

This distinction matters for interpreting the structural risk of AI coding tools. Under hypothesis (a), the threat lies in new project construction and wholesale rewrites. Under hypothesis (b), 12 months may simply be too short. Resolving this requires either estimating AI-authored LOC fractions or observing longer time horizons.

7.5 Connection to Attestation

Fan-in topology offers a passive structural signal that complements behavioral testing. Structural metrics would require building the intermediate abstraction layer to replicate the log-normal fan-in signature of a genuinely maintained codebase. Gini and z_{lr} could serve as lightweight health indicators in CI pipelines or code review tools, flagging structural flattening before it accumulates.

7.6 Limitations

Fan-in is first-order; it ignores call graphs, data flow, and semantic coupling.

Agentic repo selection is biased toward projects with public AI attribution; private vibe-coded codebases may differ.

Shallow cloning captures HEAD without evolutionary trajectory for the cross-sectional study.

FastAPI's design philosophy (thin routers, heavy external dependencies) may independently reduce intra-repo coupling regardless of authorship. Of the 22 Cohort B repos, at least 3 are FastAPI-style backends (dark-factory-experiment, ott-platform, both Gini=0.000, and codebase-mcp). If further repos in the fitted group are also FastAPI-style, the confound may account for a portion of the Gini gap not attributable to AI authorship.

The longitudinal study uses N=2 repos (effectively N=1 with a full post-adoption window). Without a matched control group (pre-2023 repos that did *not* adopt AI tools over the same period), we cannot rule out that the Celery and Django Gini trajectories simply reflect continued natural maturation. The Gini increase we observe may have nothing to do with AI adoption at all. We report these preliminary results to establish the longitudinal methodology and provide a baseline for future studies with longer post-adoption windows and larger repo cohorts.

The z_{lr} statistic uses OLS residuals as proxy log-likelihoods, not the MLE-based Vuong (1989) test; see Section 3.3.

---

8. Conclusion

We measured import fan-in distributions across 15 mature human-written and 22 AI-generated Python repositories, and tracked fan-in topology monthly in 2 repos before and after documented AI adoption. Human-written repos show log-normal fan-in with Gini mean 0.882, consistent with Constructal flow optimization by analogy. AI-generated repos exhibit two forms of structural degeneration: total isolation (Gini=0.000, 3/22 repos) and partial flattening (Gini mean 0.725 among measurable repos, p=0.0011, r=-0.744). 11 of 12 fitted agentic repos retain log-normal shape, but with significantly weaker parabolic curvature (z_{lr} mean 2.77 vs 6.00, p=0.019). The log-normal signature is not erased, it is attenuated.

The longitudinal pilot finds no Gini decline in mature repos following AI adoption on a 12-month horizon (Celery Gini +0.0016, Django +0.0005). With N=1 effective repo and no matched control, the pilot is insufficient to adjudicate between a structural-genesis hypothesis (flattening confined to new-project construction) and a null AI-adoption effect on mature codebases. We report it as preliminary methodology, not validation.

Fan-in Gini is a cheap, CI-friendly structural health metric. A project with Gini below 0.75 and weak log-normal signal is structurally flat. For monitoring established repos over time, z_{lr} and $delta_{AIC}$ are faster-moving and more sensitive to changes in the middle of the distribution where the constructal hierarchy lives.

References

- Barabasi, A.L. & Albert, R. (1999). Emergence of scaling in random networks. *Science*, 286, 509-512.
- Bejan, A. (1997). Constructal-theory network of conducting paths for cooling a heat generating volume. *Int. J. Heat Mass Transfer*, 40(4), 799-811. doi:10.1016/0017-9310(96)00175-5
- Bejan, A. & Lorente, S. (2011). The constructal law and the evolution of design in nature. *Physics of Life Reviews*, 8(3), 209-240.
- Burnham, K.P. & Anderson, D.R. (2002). *Model Selection and Multimodel Inference*. Springer.
- Clauset, A., Shalizi, C.R. & Newman, M.E.J. (2009). Power-law distributions in empirical data. *SIAM Review*, 51(4),

661-703.

- Decan, A., Mens, T. & Grosjean, P. (2019). An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1), 381-422. arXiv:1710.04936.
- Hassan, A.E. & Holt, R.C. (2004). Predicting change propagation in software systems. *Proc. ICSM 2004*, 284-293.
- Hess, W.R. (1917). Uber die periphere Regulierung der Blutzirkulation. *Pflugers Archiv*, 168, 439-490.
- Killick, R., Fearnhead, P. & Eckley, I.A. (2012). Optimal detection of changepoints with a linear computational cost. *JASA*, 107(500), 1590-1598.
- Maillart, T. & Sornette, D. (2008). Empirical tests of Zipf's law mechanism in open source Linux distribution. *Physical Review Letters*, 101, 218701.
- Mao, T., Zhao, D., Tang, H., Wang, X. & Zhang, H. (2026). A large-scale empirical study of AI-generated code in real-world repositories. *arXiv:2603.27130*.
- Murray, C.D. (1926). The physiological principle of minimum work: I. The vascular system and the cost of blood volume. *PNAS*, 12(3), 207-214.
- Nagappan, N., Ball, T. & Zeller, A. (2005). Mining metrics to predict component failures. *Proc. ICSE 2005*, 452-461.
- Paipuru, T. (2026). CodeCompass: Navigating the Navigation Paradox in Agentic Code Intelligence. *arXiv:2602.20048*.
- Vuong, Q.H. (1989). Likelihood ratio tests for model selection and non-nested hypotheses. *Econometrica*, 57(2), 307-333.